LAB 04 – DANH SÁCH SẢN PHẨM VỚI DỮ LIỆU DUMMY

4.1. SỬ DỤNG TIÊU CHUẨN CODE HÓA

Tiêu chuẩn code hóa là một bộ quy tắc và thỏa thuận được sử dụng khi viết code nguồn bằng một ngôn ngữ lập trình cụ thể. Việc sử dụng tiêu chuẩn code hóa mang lại một số lợi ích, chẳng hạn như:

- Nó mang lại vẻ ngoài thống nhất cho các code được viết bởi các lập trình viên khác nhau.
- Nó cải thiện khả năng đọc và bảo trì code và giảm độ phức tạp.
- Nó giúp tái sử dụng code và giúp phát hiện lỗi dễ dàng.
- Nó thúc đẩy việc thực hành lập trình tốt và tăng hiệu quả của người lập trình.

Một trong những tiêu chuẩn code hóa PHP được sử dụng nhiều nhất là PSR-2. Đây là một tiêu chuẩn được thiết lập bởi PHP-FIG (PHP Framework Interop Group, có thể tìm thêm thông tin về PSR-2 tại https://www.php-fig.org/psr/psr-2/). Một số công cụ giúp lập trình viên tự động kiểm tra code của họ theo các tiêu chuẩn code hóa này. Trong hướng dẫn này, chúng ta sẽ sử dụng PHP_CodeSniffer.

4.1.1. GIỚI THIỆU PHP_CODESNIFFER

PHP_CodeSniffer là một bộ gồm hai tập lệnh PHP. Tập lệnh phpcs code hóa các tập tin PHP, JavaScript và CSS để phát hiện các vi phạm dựa trên tiêu chuẩn code hóa đã xác định. Tập lệnh phpcbf tự động sửa các vi phạm tiêu chuẩn code hóa. PHP_CodeSniffer là một công cụ phát triển thiết yếu giúp đảm bảo code luôn rõ ràng và nhất quán. Có thể tìm thêm thông tin về PHP_CodeSniffer tại: https://github.com/squizlabs/PHP_CodeSniffer.

4.1.2. CÀI ĐẶT VÀ CẤU HÌNH PHP_CODESNIFFER

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
composer require --dev squizlabs/php_codesniffer
```

Lệnh trên cài đặt thư viện PHP_CodeSniffer trong tập tin composer.json, lệnh này sẽ tải xuống và trích xuất các tập tin thư viện trong thư mục nhà cung cấp.

Chúng ta sẽ áp dụng cấu hình nhanh của PHP_CodeSniffer. Trong thư mục gốc của dự án, tạo một tập tin mới phpcs.xml và thêm code sau vào đó.

Code

```
<?xml version="1.0"?>
<ruleset name="PHP_CodeSniffer">
    <description>The coding standard for the Online Store project</description>
    <rule ref="PSR2"/>
    <file>app/</file>
    <file>config/</file>
    <file>database/</file>
    <file>routes/</file>
    <exclude-pattern>*/migrations/*</exclude-pattern>
</ruleset>
```

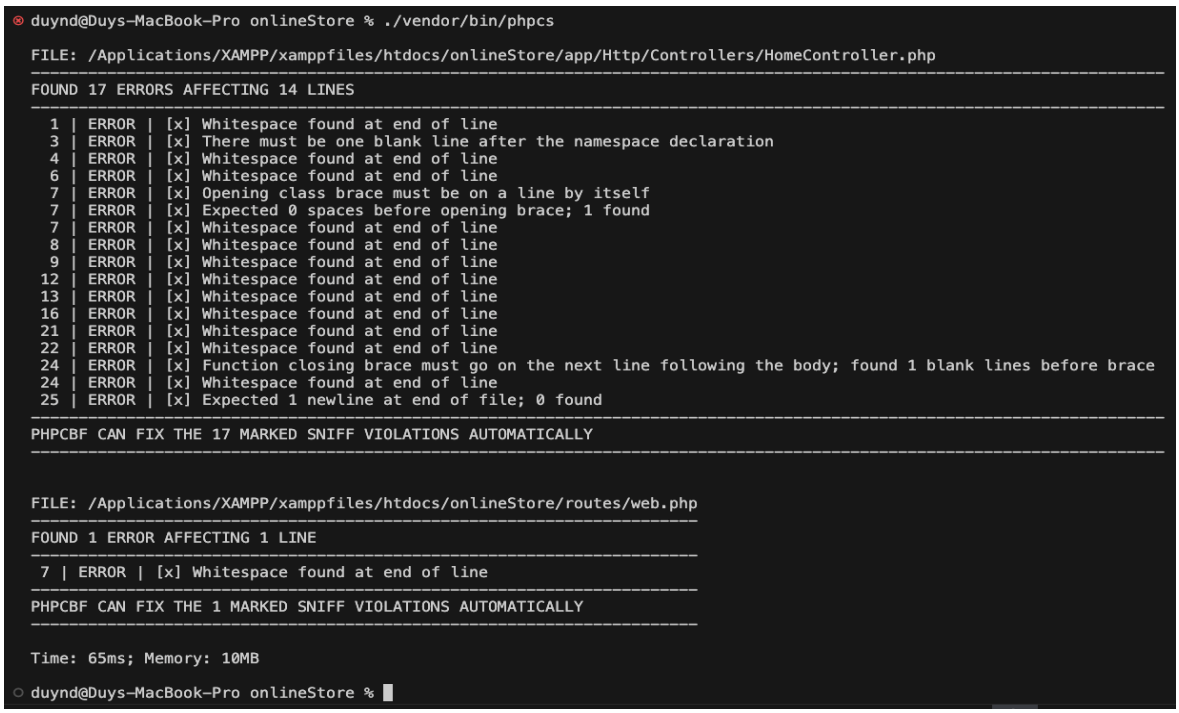
Tập tin này thiết lập rằng PHP_CodeSniffer sẽ sử dụng PSR-2 làm tiêu chuẩn kiểu code hóa. Nó cũng định nghĩa rằng khi chúng ta chạy PHP_CodeSniffer, nó sẽ tìm thấy lỗi trong các thư mục app/*, config/*, database/*, và routes/*. Chúng ta loại trừ các vị trí có thư mục migrations.

Chạy PHP_CodeSniffer

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
./vendor/bin/phpcs
```



Hình 4-1. Chạy PHP CodeSniffer.

Lệnh trên thực thi PHP_CodeSniffer dựa trên cấu hình tùy chỉnh được xác định trong tập tin phpcs.xml (xem Hình 4-1). Trường hợp của chúng ta cho thấy 17 lỗi trong tập tin HomeController, tất cả đều có dấu "X", có nghĩa là PHP_CodeSniffer có thể xử lý và sửa chúng (nếu muốn). Một số lỗi đó có liên quan đến khoảng trắng, thụt lề và các quy tắc được xác định trong hướng dẫn về kiểu code hóa PSR2. Ví dụ: có một dòng trống khó chịu ở cuối phương thức about của HomeController (trước khi đóng dấu ngoặc nhọn). Vì vậy, PHP_CodeSniffer đã đánh dấu nó là lỗi.

Để tự động sửa nó, chúng ta cần thực thi tập tin phpcbf PHP_CodeSniffer bằng lệnh sau (xem Hình 4-2).

Terminal

```
./vendor/bin/phpcbf
```

Sau đó, nó hiển thị các lỗi đã được sửa. Chiến lược này tốt hơn so với sửa chữa thủ công. Trong phần còn lại, chúng ta sẽ sử dụng PHP_CodeSniffer để cải thiện chất lượng code.

```
duynd@Duys-MacBook-Pro onlineStore % ./vendor/bin/phpcbf

PHPCBF RESULT SUMMARY
----------------------------------------------------------------------------------------------------
FILE                                                                                  FIXED  REMAINING
----------------------------------------------------------------------------------------------------
/Applications/XAMPP/xamppfiles/htdocs/onlineStore/app/Http/Controllers/HomeController.php     17     0
/Applications/XAMPP/xamppfiles/htdocs/onlineStore/routes/web.php                              1      0
----------------------------------------------------------------------------------------------------
A TOTAL OF 18 ERRORS WERE FIXED IN 2 FILES
----------------------------------------------------------------------------------------------------

Time: 80ms; Memory: 10MB
```

Hình 4-2. Sửa lỗi style bằng PHP CodeSniffer.

4.1.3. CHÚ THÍCH

Có những công cụ khác có thể sử dụng để mở rộng khả năng của PHP_CodeSniffer. Ví dụ, Larastan. Larastan là một công cụ phân tích tĩnh cho các dự án Laravel (https://github.com/nunomaduro/larastan), công cụ này thậm chí còn phát hiện ra các lỗi trong code mà không cần chạy nó.

PHP_CodeSniffer không hoạt động tốt với các tập tin blade. Một plugin định dạng Visual Studio Code có tên là Format HTML in PHP có thể giúp bạn định dạng các view Blade.

PHP_CodeSniffer không tự động sửa tất cả các lỗi. Ví dụ: có một tiêu chuẩn sử dụng kiểu code hóa CamelCase để đặt tên cho các phương thức. Nếu thay đổi phương thức HomeController about thành About, PHP_CodeSniffer sẽ đánh dấu lỗi đó nhưng nó sẽ

không tự động sửa lỗi, phải sửa lỗi này bằng tay.

Có một câu chuyện hay về "The Broken Window Theory - Lý thuyết cửa sổ vỡ" được mô tả trong cuốn sách (2019 - Thomas, D., & Hunt, A. - The Pragmatic Programmer: your Journey to Mastery), có thể tìm kiếm nó trong Google. Từ câu chuyện đó, chúng ta muốn nêu bật mẹo tiếp theo. Đừng để "cửa sổ bị hỏng" (ví dụ: thiết kế tồi, quyết định sai hoặc code kém) chưa được sửa chữa. Hãy sửa chữa từng cái một ngay khi nó được phát hiện. Các phần trước cho thấy nhiều cửa sổ bị hỏng, may mắn thay đã được sửa chữa.

Luôn sử dụng công cụ tiêu chuẩn code hóa, bộ định dạng, công cụ phân tích code tĩnh hoặc thậm chí là kết hợp chúng trong dự án. Nó sẽ giúp tiết kiệm nhiều thời gian và cải thiện chất lượng code. Ngoài ra, sẽ tìm thấy linters có sẵn cho hầu hết các ngôn ngữ lập trình. Ngoài ra, hãy đưa quy tắc vào tài liệu quy tắc kiến trúc đề cập rằng tất cả thay đổi code phải được xác minh trước bằng cách sử dụng các công cụ này. Thậm chí có thể tự động hóa quy trình này (với chiến lược quy trình hoặc CI/CD).

4.2. LIST PRODUCTS VỚI DUMMY DATA

Trong phần này, chúng ta sẽ triển khai chức năng liệt kê tất cả các sản phẩm và có thể nhấp vào các sản phẩm đó và hiển thị dữ liệu của chúng trong một phần riêng biệt.

4.2.1. SỬA ĐỔI TUYẾN (MODIFYING ROUTES)

Hãy bắt đầu bằng cách đưa vào một số tuyến để liệt kê sản phẩm và hiển thị dữ liệu của một sản phẩm. Trong Routes/web.php, hãy thay đổi như sau (phần in đậm, màu đỏ).

Code

```
...
Route::get('/', 'App\Http\Controllers\HomeController@index')-
>name("home.index");

Route::get('/about', 'App\Http\Controllers\HomeController@about')-
>name("home.about");

Route::get('/products', 'App\Http\Controllers\ProductController@index')-
>name("product.index");

Route::get('/products/{id}', 'App\Http\Controllers\ProductController@show')-
>name("product.show");
```

Chúng ta sẽ liệt kê tất cả các sản phẩm ứng dụng trong tuyến đầu tiên ("/products"). Tuyến thứ hai sẽ được sử dụng để hiển thị dữ liệu của một sản phẩm. "/products/{id}" nhận tham số tên là id. Tham số này là id sản phẩm để xác định dữ liệu sản phẩm nào sẽ hiển thị. Ví dụ: nếu chúng ta truy cập "/products/1", ứng dụng sẽ hiển thị dữ liệu của sản phẩm có id=1.

Ở đây, cả hai tuyến đều được kết nối với ProductController. Vì vậy, hãy thực hiện nó.

4.2.2. PRODUCTCONTROLLER

Trong app/Http/Controllers, tạo một tập tin mới ProductController.php và điền code sau.

Code

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public static $products = [
        ["id"=>"1", "name"=>"TV", "description"=>"Best TV", "image" =>
"TV.jpg", "price"=>"2000"],
        ["id"=>"2", "name"=>"iPhone", "description"=>"Best iPhone", "image" =>
"iPhone.jpeg", "price"=>"1500"],
        ["id"=>"3", "name"=>"Chromecast", "description"=>"Best Chromecast",
"image" => "Chromecast.jpeg", "price"=>"300"],
        ["id"=>"4", "name"=>"Glasses", "description"=>"Best Glasses", "image"
=> "Glasses.jpeg", "price"=>"500"]
    ];
    public function index()
    {
        $viewData = [];
        $viewData["title"] = "Products - Online Store";
        $viewData["subtitle"] = "Danh sách sản phẩm.";
        $viewData["products"] = ProductController::$products;
        return view('product.index')->with("viewData", $viewData);
    }
    public function show($id)
    {
        $viewData = [];
        $product = ProductController::$products[$id-1];
        $viewData["title"] = $product["name"]." - Online Store";
        $viewData["subtitle"] = $product["name"]." – Thông tin sản phẩm.";
        $viewData["product"] = $product;
        return view('product.show')->with("viewData", $viewData);
    }
}
```

Hãy phân tích code trước đó theo từng phần.

Code

```
    public static $products = [
        ["id"=>"1", "name"=>"TV", "description"=>"Best TV", "image" => "TV.jpg",
"price"=>"2000"],
        ["id"=>"2", "name"=>"iPhone", "description"=>"Best iPhone", "image" =>
"iPhone.jpeg", "price"=>"1500"],
        ["id"=>"3", "name"=>"Chromecast", "description"=>"Best Chromecast", "image" =>
"Chromecast.jpeg", "price"=>"300"],
        ["id"=>"4", "name"=>"Glasses", "description"=>"Best Glasses", "image" =>
"Glasses.jpeg", "price"=>"500"]
    ];
```

$products là một mảng chứa tập hợp các sản phẩm cùng với dữ liệu của nó. Trong chỉ số mảng 0, chúng ta có tích có id=1 ("TV"). Chúng ta có bốn sản phẩm giả. Hiện tại, chúng được lưu trữ dưới dạng thuộc tính tĩnh của lớp ProductController. Sau này chúng ta sẽ truy xuất dữ liệu sản phẩm từ cơ sở dữ liệu MySQL trong các phần sắp tới.

Code

```
    public function index()
    {
        $viewData = [];
        $viewData["title"] = "Products - Online Store";
        $viewData["subtitle"] = "Danh sách sản phẩm.";
        $viewData["products"] = ProductController::$products;
        return view('product.index')->with("viewData", $viewData);
    }
```

Phương thức index lấy mảng sản phẩm và gửi chúng đến view product.index để hiển thị.

Code

```
public function show($id) {
     $viewData = [];
     $product = ProductController::$products[$id-1];
     $viewData["title"] = $product["name"]." - Online Store";
     $viewData["subtitle"] = $product["name"]." – Thông tin sản phẩm.";
     $viewData["product"] = $product;
     return view('product.show')->with("viewData", $viewData);
}
```

Phương thức show lấy $id làm tham số (id được thu thập từ URL). Chúng ta trích xuất dữ liệu sản phẩm có id đó (kiểm tra dòng in đậm, màu đỏ). Chúng ta trừ một đơn vị vì chúng ta đã lưu trữ sản phẩm có id=1 trong chỉ mục mảng ProductController::$products là 0, sản

phẩm có id=2 trong chỉ mục mảng ProductController::$products là 1, v.v. Sau đó, chúng ta gửi dữ liệu sản phẩm đến view product.show.

4.2.3. PRODUCT VIEWS

Trước tiên, hãy triển khai view product.index và sau đó là view product.show.

Product index view

Trong resources/views/, tạo thư mục con product. Sau đó, trong resources/views/product, tạo một tập tin mới index.blade.php và điền code sau vào đó.

Code

```
@extends('layouts.app')
@section('title', $viewData["title"])
@section('subtitle', $viewData["subtitle"])
@section('content')
<div class="row">
    @foreach ($viewData["products"] as $product)
        <div class="col-md-4 col-lg-3 mb-2">
            <div class="card">
                <img src="{{ asset('/img/'.$product["image"]) }}" class="card-
img-top img-card">
                <div class="card-body text-center">
                    <a href="{{ route('product.show', ['id'=> $product["id"]])
}}" class="btn bg-primary text-white">{{ $product["name"] }}</a>
                </div>
            </div>
        </div>
    @endforeach
</div>
@endsection
```

Phần quan trọng của code trên đó là @foreach. @foreach là một lệnh Blade cho phép chúng ta lặp lại một danh sách. Chúng ta lặp lại qua từng sản phẩm và hiển thị hình ảnh cũng như tên sản phẩm. Có thể tìm thêm thông tin về các chỉ thị của Blade tại đây: https://laravel.com/docs/11.x/blade#blade-directives.

Code

```
<a href="{{ route('product.show', ['id'=> $product["id"]]) }}" class="btn bg-
primary text-white">{{ $product["name"] }}</a>
```

Cuối cùng, chúng ta đặt một liên kết đến tên sản phẩm. Liên kết sẽ định tuyến đến tuyến

product.show (được xác định trước đó trong routes/web.php) và nó yêu cầu gửi một tham số. Trong trường hợp này, chúng ta gửi id sản phẩm của sản phẩm lặp lại hiện tại.

Product show view

Trong resources/views/product, tạo một tập tin mới show.blade.php và điền code sau vào.

Code

```
@extends('layouts.app')
@section('title', $viewData["title"])
@section('subtitle', $viewData["subtitle"])
@section('content')
<div class="card mb-3">
    <div class="row g-0">
        <div class="col-md-4">
            <img src="{{ asset('/img/'.$viewData["product"]["image"]) }}"
class="img-fluid rounded-start">
        </div>
        <div class="col-md-8">
            <div class="card-body">
                <h5 class="card-title">
                    {{ $viewData["product"]["name"] }} (${{
$viewData["product"]["price"] }})
                </h5>
                <p class="card-text">{{ $viewData["product"]["description"]
}}</p>
                <p class="card-text"><small class="text-muted">Add to
Cart</small></p>
            </div>
        </div>
    </div>
</div>
@endsection
```

Chúng ta hiển thị hình ảnh, tên và mô tả sản phẩm trong đoạn code trên. Hãy nhớ rằng, chúng ta đang sử dụng các sản phẩm giả. Điều này sẽ thay đổi trong các phần sau.

Trong các ví dụ trước, chúng ta đã xác định cấu trúc để lưu trữ controllers, phương thức của controllers, tên tuyến và view. Ví dụ: tuyến product.show được liên kết với phương thức hiển thị ProductController, hiển thị view product/show. Cố gắng sử dụng chiến lược này trên toàn bộ dự án vì nó tạo điều kiện thuận lợi cho việc tìm kiếm các quan điểm về phương thức của controllers tương ứng và ngược lại.

4.2.4. CẬP NHẬT LIÊN KẾT TRONG HEADER

Bây giờ, hãy đưa liên kết sản phẩm vào tiêu đề. Trong

resources/views/layouts/app.blade.php hãy in thay đổi phần in đậm, màu đỏ sau.

Code

```
<!doctype html>
...
     <!-- header -->
     <nav class="navbar navbar-expand-lg navbar-dark bg-secondary py-4">
          <div class="container">
            <a class="navbar-brand" href="#">Online Store</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNavAltMarkup" aria-
controls="navbarNavAltMarkup" aria-expanded="false" aria-label="Toggle
navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNavAltMarkup">
                <div class="navbar-nav ms-auto">
                    <a class="nav-link active" href="{{ route('home.index')
}}">Home</a>
                    <a class="nav-link active" href="{{ route('product.index')
}}">Products</a>
                    <a class="nav-link active" href="{{ route('home.about')
}}">About</a>
                </div>
            </div>
        </div>
    </nav>
    <header class="masthead bg-primary text-white text-center py-4">
        <div class="container d-flex align-items-center flex-column">
            <h2>@yield('subtitle', 'Online Store - Laravel Framework')</h2>
        </div>
    </header>
    <!-- header -->
...
```

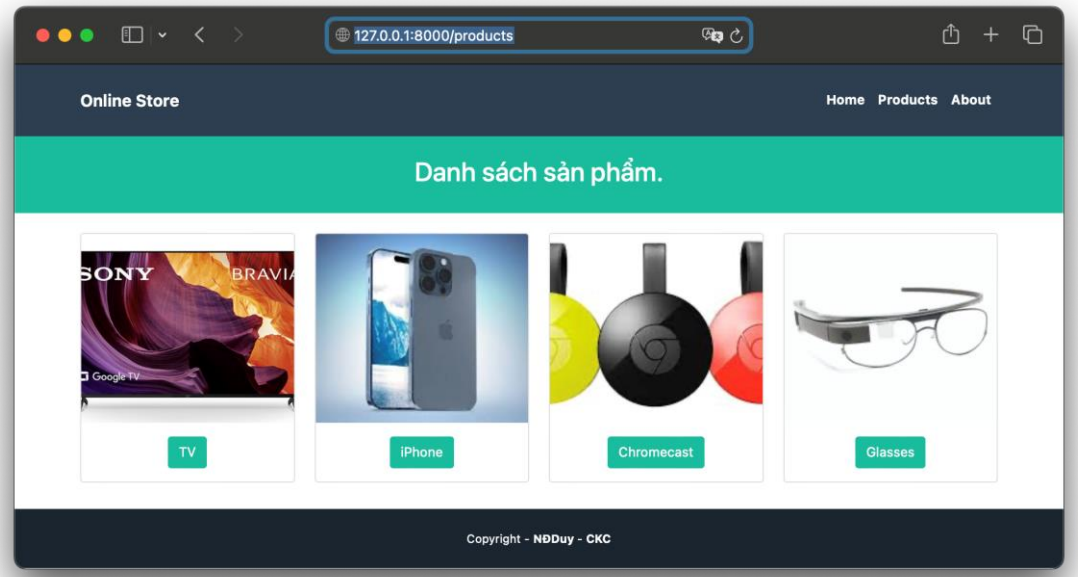
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

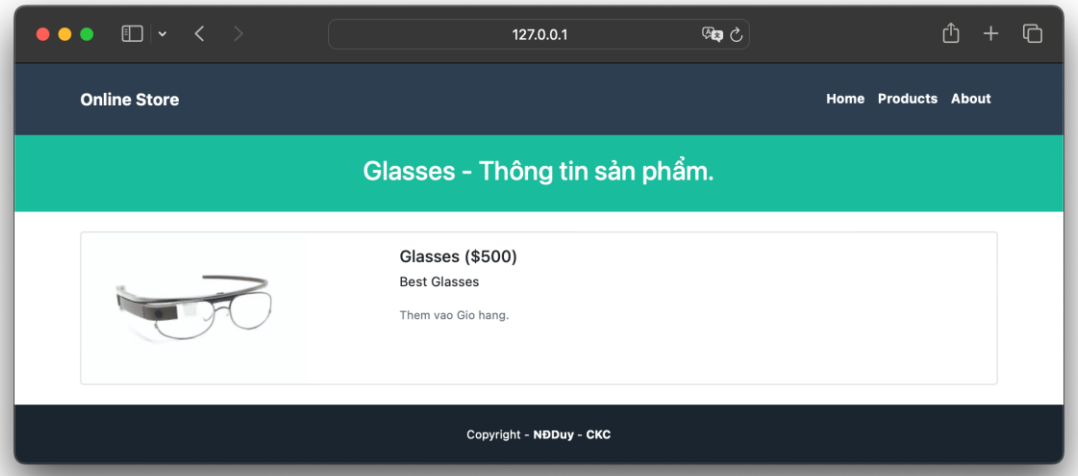
Terminal

```
php artisan serve
```

Bây giờ, có thể truy cập trang Products (xem Hình 4-3) và điều hướng đến một sản phẩm cụ thể (xem Hình 4-4).



Hình 4-3. Online Store – Trang Products.



Hình 4-4. Online Store – Trang Thông tin sản phẩm.